

Personal IT Homelab - Final Tech Doc

1. Project Overview

This project is a personal IT homelab built to gain practical experience with **virtualization, Linux administration, networking, Docker, self-hosted services, VPNs, DNS, HTTPS, Windows Server, Active Directory, Group Policy, and enterprise-style troubleshooting.**

The lab is hosted on a Dell OptiPlex 5080 Micro running Proxmox VE. Proxmox provides the virtualization layer for Linux and Windows virtual machines and lightweight LXC containers.

The environment is intentionally designed to remain **private and inaccessible from the public Internet.** Remote access is provided through Tailscale, while Pi-hole provides DNS filtering and local name resolution, and Caddy provides HTTPS and reverse proxying for applications.

The current architecture combines:

- Bare-metal virtualization with Proxmox
- Ubuntu Server and Docker
- Debian LXC infrastructure services
- Self-hosted media/file applications
- Internal DNS and ad blocking
- Private VPN access
- Internal HTTPS using a private certificate authority
- Service monitoring
- A custom web dashboard
- Windows Server Active Directory
- Windows 11 domain workstation
- Group Policy and security filtering
- Active Directory account administration and troubleshooting

2. Physical Hardware

Homelab Server

Dell OptiPlex 5080 Micro

Component	Configuration
CPU	Intel Core i5-10500T
CPU topology	6 cores / 12 threads
RAM	32 GB DDR4
Storage	~465 GB / 500 GB Kingston NVMe SSD
Network	Ethernet
Operating system	Proxmox VE

The OptiPlex was repurposed from a normal Windows desktop into a dedicated virtualization server.

RAM Upgrade

The system originally contained:

- 16 GB DDR4-2400 SO-DIMM
- One populated memory slot

A second 16 GB DDR4 SO-DIMM was installed, bringing the system to:

- 32 GB total RAM
- Two populated SO-DIMM slots

The additional module was DDR4-3200, but the system automatically operated the memory at a compatible 2400 MT/s speed.

Hardware validation confirmed:

- Correct DDR4 standard
- Correct SO-DIMM form factor
- Correct 1.2 V voltage
- BIOS detected the memory change
- Windows recognized 32 GB
- Both memory slots were populated

Reason for Upgrade

The additional RAM increased capacity for running multiple workloads simultaneously, including:

- Proxmox
- Ubuntu Server
- Docker containers
- Windows Server
- Windows 11
- Infrastructure LXC's

This provided substantially more flexibility than the original 16 GB configuration.

3. Proxmox VE

Installation

Proxmox VE was installed directly onto the Dell OptiPlex as the **bare-metal hypervisor**.

Before installation:

- UEFI boot was verified
- Intel Virtualization Technology was enabled
- Secure Boot configuration was reviewed
- A Proxmox installation USB was created
- Ethernet connectivity was verified

The original Windows installation and existing partitions on the NVMe drive were replaced by Proxmox.

Storage

The primary NVMe device was identified as: KINGSTON SNV2S500G

Capacity reported by the installer: 465.76 GiB

Proxmox was installed using an **ext4-based configuration**, with Proxmox's storage architecture providing separate storage locations for system data, ISOs, templates, backups, and VM/container disks.

Host Identity

The Proxmox host was configured as:

Hostname: pve.homelab

Short hostname: pve

IP: 10.0.0.50/24

Gateway: 10.0.0.1

DNS: 75.75.75.75

The Proxmox management interface is available at: <https://10.0.0.50:8006>

Proxmox is administered remotely through its web interface rather than requiring a dedicated monitor and keyboard.

4. Proxmox Networking

The home network is provided by an Xfinity XB8 gateway.

Router/Gateway: 10.0.0.1

LAN: 10.0.0.0/24

The Proxmox host has a permanent network identity through both a static configuration and a DHCP reservation on the router.

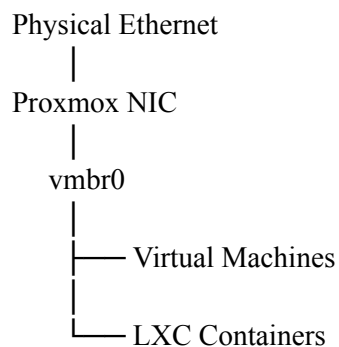
pve.homelab

10.0.0.50

Linux Bridge

Proxmox uses a Linux bridge named: vmbbr0

The architecture is:



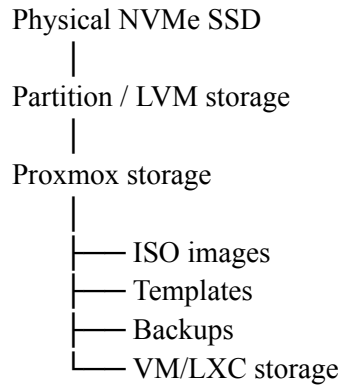
vmbbr0 functions similarly to a virtual switch, allowing virtual machines and containers to communicate with the physical LAN.

This allows virtual systems to behave like independent devices on the home network.

5. Proxmox Storage Architecture

The physical storage is provided by the Kingston NVMe SSD.

Conceptually:



The primary Proxmox storage locations include:

local

Used for items such as:

- ISO images
- Templates
- Backups

local-lvm

Used primarily for:

- VM virtual disks
- Container storage

Understanding the distinction between physical disks, partitions, LVM, Proxmox storage pools, and virtual disks was an important part of learning the virtualization architecture.

6. Proxmox Maintenance

After installation:

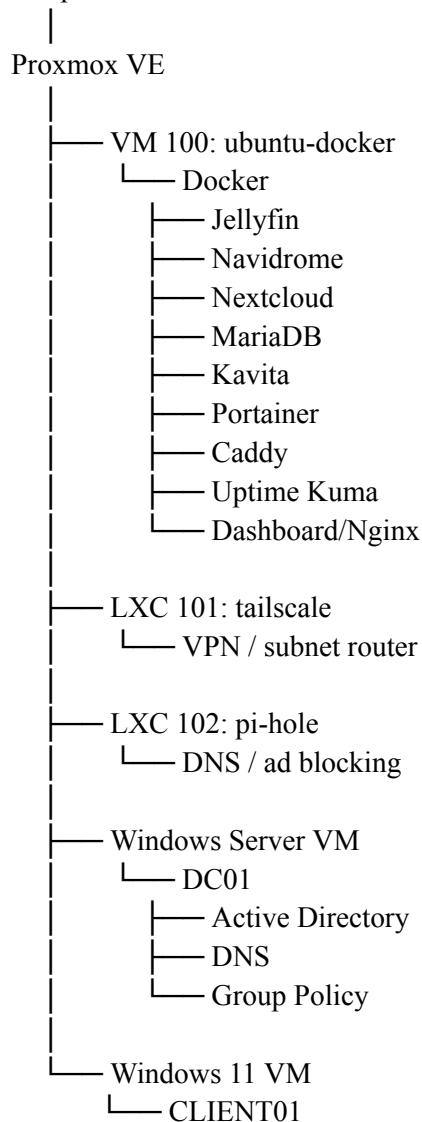
- The enterprise repository was disabled
- The community/no-subscription repository was enabled
- Package repositories were refreshed
- Proxmox system updates were installed

The system was then ready to host the rest of the homelab.

7. Virtualization Architecture

The current high-level architecture is:

Dell OptiPlex 5080 Micro



The architecture uses **VMs when full operating-system isolation is useful** and **LXC containers for lightweight infrastructure services**.

8. Ubuntu Docker VM

VM Configuration

The primary application server is:

VM ID: 100

Name: ubuntu-docker

Virtual hardware:

Resource	Configuration
OS	Ubuntu Server 26.04 LTS
Firmware	UEFI / OVMF
Chipset	q35
CPU	2 cores
RAM	8 GiB
Disk	80 GiB
Storage controller	VirtIO SCSI
Network	VirtIO
Network bridge	vmbr0
QEMU Guest Agent	Enabled
Secure Boot	Enabled

The VM's virtual disk uses Proxmox **local-lvm**.

Ubuntu was installed using:

- EFI partition

- /boot
- LVM partition
- ubuntu-vg
- ubuntu-lv

The VM's storage is separate from the Proxmox host's operating system.

9. Ubuntu Networking

The Ubuntu VM uses a VirtIO network adapter, which appears inside Ubuntu as: ens18

The VM receives a permanent DHCP reservation: ubuntu-docker → 10.0.0.60

Network path:

```
graph TD
    A[Ubuntu VM] --> B[VirtIO NIC]
    B --> C[vmbr0]
    C --> D[Proxmox physical NIC]
    D --> E[Xfinity router]
```

The VM therefore behaves like a normal physical device on the home LAN.

10. Ubuntu Remote Administration

OpenSSH Server was installed and configured.

Initial SSH access used password authentication, after which SSH key authentication was configured.

My MacBook was configured with an SSH shortcut allowing the server to be accessed using:

ssh ubuntu-docker

Passwordless SSH authentication was verified successfully.

This provides secure remote administration without requiring a graphical interface or physical access to the server.

11. Docker

Docker Engine was installed on Ubuntu.

Configured:

- Docker Engine
- Docker Compose
- Docker service enabled at boot
- Docker service running
- **evan** added to the **docker** group

Docker commands were verified to work without **sudo**.

The Docker environment is organized under: ~/homelab/

Each application has its own directory and Docker Compose configuration.

12. Portainer

Portainer CE was deployed using Docker Compose.

Purpose:

- Web-based Docker administration
- Container management
- Image management
- Volume management
- Network management
- Stack management
- Container logs
- Container lifecycle operations

Portainer is accessible through: <https://portainer.homelab>

Its Docker integration uses: /var/run/docker.sock

Portainer was verified to manage both its own environment and Compose stacks created outside of Portainer.

13. Media Storage

The application media directory is: ~/homelab/media/

Structure:

```
media/  
├── movies/  
├── music/  
└── books/
```

Movies

Movies are stored under: ~/homelab/media/movies/

Example test library included:

- Teenage Mutant Ninja Turtles (1990)
- The Terminator (1984)
- Terminator 2: Judgment Day (1991)

Music

Music is stored under: ~/homelab/media/music/

Music is organized by:

```
Artist/  
└── Album/  
    └── tracks.flac
```

The library contains artists such as:

- Pink Floyd
- Radiohead
- Tame Impala

Music files are stored as FLAC.

Important supporting files include:

- **.flac** — lossless audio
- **.lrc** — synchronized lyrics
- **cover.jpg** — album artwork

FLAC metadata was verified with **metaflac**, and Navidrome successfully used the metadata to organize the library.

Books

Books are stored in: ~/homelab/media/books/

with individual folders containing their corresponding PDF files.

14. Jellyfin

Jellyfin is the homelab's self-hosted movie/media streaming platform.

Deployment: Docker Compose

Port: 8096

Media: ~/homelab/media/movies/

The media directory is mounted into the container.

Jellyfin was configured with persistent application storage and successfully tested.

Verified functionality includes:

- Movie scanning
- Metadata retrieval
- Posters
- Genres
- Continue Watching
- Watch history/resume functionality
- Movie playback
- MKV playback

Normal access is through: <https://jellyfin.homelab>

15. Navidrome

Navidrome is the self-hosted music streaming platform.

Deployment: Docker Compose

Port: 4533

Media: ~/homelab/media/music/

Music is mounted into the container as a **read-only bind mount**.

This allows the server to access the music while preventing the application from modifying the original library.

Verified functionality:

- FLAC playback
- Album scanning
- Artist/album organization
- FLAC metadata
- Album artwork
- .lrc synchronized lyrics
- Administrator account

Normal access: <https://navidrome.homelab>

16. Nextcloud

Nextcloud provides private, self-hosted file storage and is intended primarily for personal digital journaling and files.

Deployment: Nextcloud + MariaDB

External port: 8080

Container port: 80

Persistent Docker volumes were configured for:

- Nextcloud application/data
- MariaDB database

The data remains on the Ubuntu VM rather than being stored on an external Nextcloud-hosted service.

The Nextcloud data volume is located under Docker's persistent volume storage.

Normal access: <https://nextcloud.homelab>

Nextcloud was also configured to recognize its **.homelab** hostname as a trusted domain.

17. Kavita

Kavita provides a self-hosted library interface for books and PDFs.

Deployment: Docker Compose

Port: 5000

Media: ~/homelab/media/books/

Books are organized into individual folders under: ~/homelab/media/books/

PDFs were successfully scanned and made available through the application.

Normal access: <https://kavita.homelab>

18. Tailscale VPN

Tailscale provides secure remote access to the private homelab.

A dedicated Debian LXC was created:

CT ID: 101

Hostname: tailscale

CPU: 1 core

RAM: 512 MiB

Storage: 8 GiB

LAN IP: 10.0.0.70

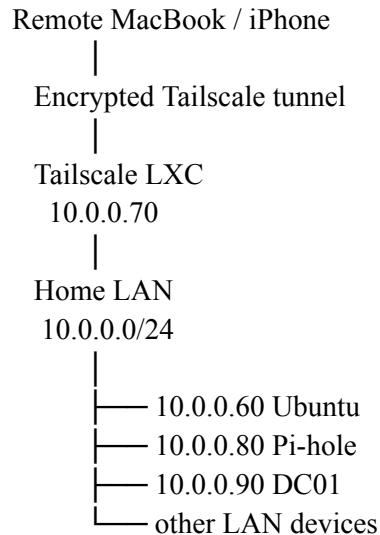
Tailscale IP: 100.115.66.70

The container is configured as a **subnet router**.

Advertised route: 10.0.0.0/24

IP forwarding was enabled for IPv4 and IPv6.

Conceptually:



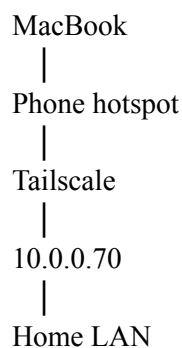
The Docker applications do not need Tailscale installed inside their containers.

The VPN operates at the network-routing layer.

19. Remote Access Testing

Remote access was successfully tested with my MacBook connected through a phone hotspot rather than the home Wi-Fi.

Traffic successfully traveled:



|
10.0.0.60
|
Docker
|
Application

This verified that the subnet router was actually providing remote access.

No router port forwarding was required.

My iPhone was also added to the Tailscale network.

20. Pi-hole

A dedicated Debian 13 LXC was created for Pi-hole.

CT ID: 102

Hostname: pi-hole

CPU: 1 core

RAM: 512 MiB

Storage: 8 GiB

IP: 10.0.0.80/24

Gateway: 10.0.0.1

Bridge: vmbr0

Pi-hole provides:

- DNS resolution
- DNS filtering
- Advertisement/tracker blocking
- Local DNS records

A static address is important because other systems need a predictable DNS server address.

21. Pi-hole DNS Filtering

Pi-hole uses Cloudflare as its upstream DNS server: 1.1.1.1

The StevenBlack Unified Hosts List, HaGeZi DNS Blocklist, and OISD Big Blocklist were enabled.

Pi-hole downloaded approximately 479,519 domains into its gravity database.

Gravity is Pi-hole's local database of domains that should be blocked.

Blocking was tested with: dig @127.0.0.1 doubleclick.net

and from the MacBook: nslookup doubleclick.net 10.0.0.80

The response returned: 0.0.0.0

This verified that Pi-hole was intercepting and blocking the DNS request.

The live query log was also used to observe DNS requests and blocked domains.

22. Tailscale + Pi-hole DNS

Tailscale was configured to use Pi-hole as the DNS server for connected Tailscale devices.

Tailscale DNS configuration:

Global nameserver: 10.0.0.80

Override local DNS: enabled

Tailscale's local DNS proxy appears to clients as: 100.100.100.100

Therefore, seeing **100.100.100.100** in **nslookup** does not mean Pi-hole is being bypassed.

The actual DNS path is:

```
MacBook
|
Tailscale DNS proxy
100.100.100.100
|
Pi-hole
10.0.0.80
|
Cloudflare
1.1.1.1
```

Remote DNS filtering was successfully tested while connected through Tailscale.

My MacBook received **0.0.0.0** for blocked domains while away from the home network.

23. Xfinity Gateway Considerations

The homelab shares the family network provided by an Xfinity XB8 gateway.

The gateway remains responsible for:

- Routing
- Internet access
- DHCP

The router was intentionally **not modified to force every device on the household network to use Pi-hole**.

Instead, Pi-hole is used primarily by the homelab/Tailscale environment.

This avoids disrupting other household devices.

24. Caddy Reverse Proxy

Caddy runs as a Docker container on the Ubuntu VM.

Directory: ~/homelab/caddy/

Caddy provides:

- Reverse proxying
- Friendly hostnames
- HTTPS
- Internal certificate management
- HTTP → HTTPS redirects

Instead of users remembering ports such as:

10.0.0.60:8096

10.0.0.60:5000

10.0.0.60:8080

They can use:

<https://jellyfin.homelab>

https://kavita.homelab
https://nextcloud.homelab

25. Internal HTTPS / Private Certificate Authority

Caddy uses: `tls internal`

This causes Caddy to create certificates using its own private internal certificate authority.

The Caddy root certificate is stored within Caddy's data directory.

The root certificate was installed and trusted on personal devices including:

- My MacBook
- My iPhone

Because the certificates are issued by a private CA rather than a public certificate authority such as *Let's Encrypt*, client devices must explicitly trust the Caddy root CA.

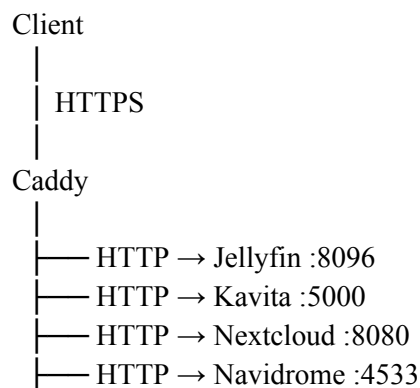
Security Distinction

The **root certificate may be distributed to trusted devices** so they can verify certificates.

The **private key for the certificate authority must never be distributed.**

26. Caddy Application Routing

The current reverse proxy architecture is:



└─ HTTPS → Portainer :9443

Portainer is different because its backend already provides HTTPS.

Caddy therefore connects to Portainer over HTTPS and is configured to skip verification of Portainer's internal certificate.

This does **not** disable encryption; it only disables certificate verification for the internal Caddy-to-Portainer connection.

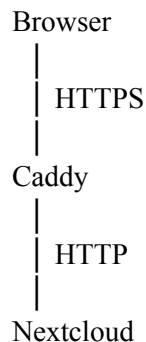
27. Nextcloud HTTPS Configuration

Nextcloud initially rejected its **.homelab** hostname because the hostname was not listed as trusted.

The configuration was updated to trust: nextcloud.homelab

Nextcloud also needed to understand that clients connect through HTTPS even though Caddy communicates with the internal HTTP endpoint.

The architecture is:



Caddy is responsible for the external TLS connection while Nextcloud is configured for the external hostname/protocol.

28. Uptime Kuma

Uptime Kuma was added to provide service monitoring.

Directory: ~/homelab/uptime-kuma/

Port: 3001

The service uses Docker Compose and SQLite.

Uptime Kuma provides availability monitoring for services using protocols such as:

- HTTP/HTTPS
- TCP
- Ping
- DNS

This differs from Portainer:

Portainer:

"What is happening inside my Docker environment?"

Uptime Kuma:

"Are my services actually reachable and working?"

29. Uptime Kuma TLS Troubleshooting

Uptime Kuma initially reported Caddy-backed services as down because it could not validate the internal Caddy certificate: unable to get local issuer certificate

The troubleshooting process was performed layer by layer.

DNS

Verified: getent hosts jellyfin.homelab

The hostname correctly resolved to: 10.0.0.60

DNS was also verified from inside the Kuma container.

Host Certificate Trust

The Caddy root CA was extracted from the Caddy container and installed into Ubuntu's trusted CA store.

After updating the certificate store, Ubuntu successfully verified the Caddy certificate.

Container Certificate Trust

The host's certificate store did not automatically apply to the Docker container.

The Caddy root certificate was therefore mounted into the Uptime Kuma container.

Node.js Certificate Trust

The Linux CA store inside the container was eventually confirmed to trust the certificate, but the Uptime Kuma application still experienced TLS errors.

Because Kuma runs on Node.js, the Caddy CA was explicitly provided using:

NODE_EXTRA_CA_CERTS

The container was recreated and the monitor successfully connected using: <https://jellyfin.homelab> with TLS verification enabled.

Lesson

This troubleshooting demonstrated that certificate trust can exist at multiple layers:

DNS
↓
Network connectivity
↓
Reverse proxy
↓
TLS certificate
↓
Host CA store
↓
Container CA store
↓
Application/runtime CA store

A certificate trusted by the host is not necessarily trusted by an application running inside a container.

30. Pi-hole Monitoring

A DNS record was created: pihole.homelab → 10.0.0.80

Pi-hole itself is not currently routed through Caddy.

Therefore its monitor uses HTTP: <http://pihole.homelab>

An initial HTTP 403 response demonstrated that Kuma could reach Pi-hole but the requested endpoint was not appropriate for monitoring.

The monitor was adjusted to use a suitable endpoint and verified successfully.

This reinforced the distinction between:

- DNS name resolution
- HTTP/HTTPS
- Caddy reverse proxying

Creating a DNS record does not automatically provide HTTPS.

31. Custom Homelab Dashboard

A custom **Evan's HomeLab Dashboard** website was built from scratch using:

- HTML
- CSS
- Local image assets

The dashboard uses a responsive layout.

Desktop: 2 columns × 4 rows

Smaller Screens: Single-column layout

Service cards include hover effects and a consistent dark/cartoon-style visual design.

The dashboard links to:

- Movies → Jellyfin
 - Music → Navidrome
 - Books → Kavita
 - Journal → Nextcloud
 - DNS / Ad Block → Pi-hole
 - VPN → Tailscale
 - Docker Management → Portainer
 - Monitoring → Uptime Kuma
-

32. Dashboard Deployment

The dashboard is deployed as a Docker container using Nginx.

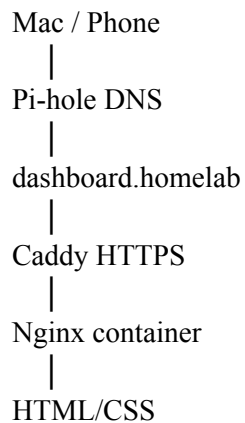
The HTML/CSS files are bind-mounted directly into the container so changes can be made without rebuilding the image.

A Pi-hole DNS record was created: dashboard.homelab → 10.0.0.60

Caddy was configured to reverse proxy the dashboard.

Final URL: https://dashboard.homelab

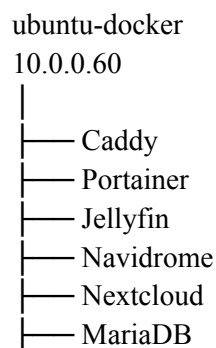
Architecture:



This project demonstrated both web development and infrastructure deployment.

33. Current Linux/Docker Application Architecture

The Ubuntu VM currently hosts:



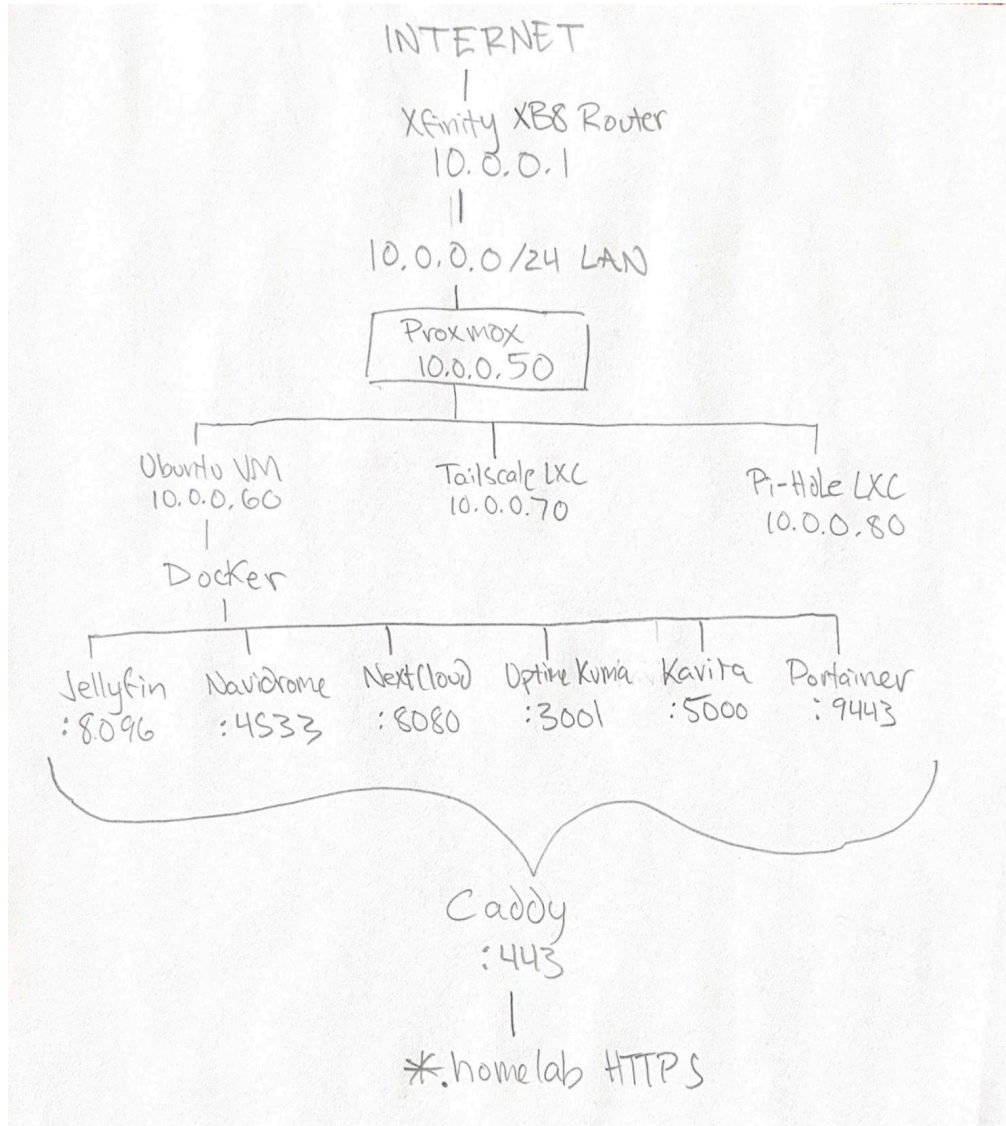
- └─ Kavita
- └─ Uptime Kuma
- └─ Nginx Dashboard

The applications are independently containerized and managed through Docker Compose.

Persistent application data is stored outside the ephemeral container lifecycle using Docker volumes or bind mounts.

34. Current Homelab Network

Ubuntu Architecture Diagram



Remote Access:

Remote Device
|
Tailscale VPN
|
Tailscale LXC
10.0.0.70
|
Home LAN

DNS:

*.homelab
|
Pi-hole
10.0.0.80
|
10.0.0.60

HTTPS:

Client
|
| HTTPS
|
Caddy
|
Docker application

35. Windows Active Directory Lab

A separate Windows environment was built inside Proxmox to practice enterprise Windows administration.

The purpose is to simulate a small organizational Windows environment containing:

- Domain controller
- DNS
- Active Directory
- Domain users
- Security groups
- Organizational Units
- Windows workstation
- Group Policy
- Local administrator management
- Account administration
- Troubleshooting

Domain: windowslab.local

NetBIOS Name: WINDOWSLAB

36. DC01 — Domain Controller

A Windows Server 2025 VM was created:

Hostname: DC01

IP: 10.0.0.90

Configuration:

Resource	Configuration
OS	Windows Server 2025
RAM	8 GB
Network	VirtIO
IP	10.0.0.90
Subnet	255.255.255.0
Gateway	10.0.0.1
DNS	127.0.0.1 / ::1

Installed roles/features:

- Active Directory Domain Services
- DNS Server
- Group Policy Management

A new forest was created: windowslab.local

DC01 acts as:

- Domain Controller
- DNS Server
- Global Catalog
- Authentication authority

The DNS delegation warning during AD installation was expected because **windowslab.local** is an internal lab domain rather than a publicly delegated DNS domain.

37. Active Directory Organizational Structure

The following OUs were created:

```
windowslab.local
├── Lab-Users
├── Lab-Groups
├── Lab-Computers
├── Lab-Servers
└── Lab-Workstations
```

Purpose:

Lab-Users: Contains domain user accounts.

Lab-Groups: Contains security groups.

Lab-Computers: General computer objects.

Lab-Servers: Server computer objects.

Lab-Workstations: Windows workstation computer objects.

An important distinction learned during the project:

OUs are organizational and management containers.

Security groups are used primarily for permissions and access control.

The OU named **Lab-Users** and the security group named **Lab-Users** are therefore completely separate AD objects.

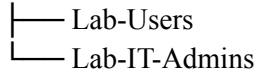
38. Active Directory Users and Security Groups

Users:

```
Lab-Users
├── Evan
└── LabAdmin
```

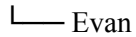
Security Groups:

Lab-Groups

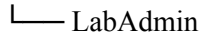


Membership:

Lab-Users



Lab-IT-Admins



LabAdmin was used for administrative testing.

The lab intentionally uses security groups rather than simply assigning every administrative user elevated domain privileges.

39. CLIENT01

A Windows 11 VM was created:

Hostname: CLIENT01

IP: 10.0.0.100

Configuration:

- Windows 11
- 8 GB RAM
- VirtIO network adapter
- vmbr0 networking

Windows initially did not recognize the VirtIO network adapter.

The appropriate VirtIO network driver was installed from the VirtIO ISO.

The resulting adapter appeared as: Red Hat VirtIO Ethernet Adapter

This demonstrated practical troubleshooting of virtual hardware and drivers.

40. CLIENT01 Network Configuration

CLIENT01 was configured with:

IP: 10.0.0.100

Subnet: 255.255.255.0

Gateway: 10.0.0.1

DNS: 10.0.0.90

IPv6 was disabled on the Ethernet adapter for simplicity in the lab.

The most important configuration is that **CLIENT01 uses DC01 as its DNS server**.

Active Directory relies heavily on DNS for locating domain services, including:

- LDAP
- Kerberos
- Domain controllers
- Other AD service records

The Xfinity router remains the default gateway but is not the DNS server used for domain operations.

41. Windows DNS Troubleshooting

Initial DNS testing revealed that Windows was still receiving Comcast/Xfinity IPv6 DNS servers:

2001:558:feed::1

2001:558:feed::2

This caused DNS queries to bypass DC01.

After disabling IPv6 on CLIENT01, DNS requests were correctly directed to: 10.0.0.90

Testing confirmed: nslookup windowslab.local

successfully resolved through DC01.

This demonstrated how incorrect DNS configuration can prevent domain functionality even when basic IP connectivity works.

42. Domain Join

CLIENT01 was successfully joined to: windowslab.local

Windows initially placed the computer object in the default: Computers container.

The computer object was manually moved into: Lab-Workstations

Final structure:

```
windowslab.local
├── Lab-Workstations
│   └── CLIENT01
```

This placement became important because workstation-targeted Group Policy was linked to the **Lab-Workstations** OU.

43. Domain Authentication

Authentication was successfully tested using the domain account: WINDOWSLAB\Evan

Commands such as: whoami

were used to verify the logged-in domain identity.

Domain membership and security group membership were also verified.

This confirmed communication between:

```
CLIENT01
|
DC01
|
Active Directory
```

44. Group Policy Architecture

Group Policy was used to manage both:

- Computer configuration

- User configuration

A critical concept learned was:

Computer Configuration follows the computer account.

User Configuration follows the user account.

Therefore:

Lab-Workstations
└─ CLIENT01
↓
Computer Configuration

while:

Lab-Users
└─ Evan
↓
User Configuration

This distinction became important during troubleshooting.

45. Lab-Workstation-Policy

GPO: Lab-Workstation-Policy

Linked To: Lab-Workstations

Purpose: General workstation configuration.

Configured Policy: Display Shutdown Event Tracker → Disabled

Because this is a **Computer Configuration** policy, it follows the workstation computer object.

Architecture:

Lab-Workstations
|
CLIENT01
|
Lab-Workstation-Policy
|

Computer Configuration

|

Shutdown Event Tracker disabled

46. Lab-Workstation-IT-Admins

GPO: Lab-Workstation-IT-Admins

Linked To: Lab-Workstations

Purpose:

Automatically give members of the **Lab-IT-Admins** security group local administrator privileges on lab workstations.

Configured Using:

Computer Configuration

→ Preferences

→ Control Panel Settings

→ Local Users and Groups

→ Local Group

Configuration:

Action: Update

Local Group: Administrators

Member: WINDOWSLAB\Lab-IT-Admins

Result:

Lab-IT-Admins

|

Lab-Workstation-IT-Admins

|

CLIENT01

|

Local Administrators

Since **LabAdmin** belongs to **Lab-IT-Admins**, **LabAdmin** automatically receives local administrator privileges on **CLIENT01**.

This demonstrates a common enterprise administration pattern:

Grant workstation administrator rights to a security group rather than individually assigning local administrator privileges to each user.

It also demonstrates that a user does not need to be a Domain Admin to administer a workstation.

47. Lab-User-Policy

GPO: Lab-User-Policy

Linked to: Lab-Users

Purpose:

Apply user-specific restrictions to members of the lab user group.

Configured:

User Configuration

→ Policies

→ Administrative Templates

→ Control Panel

→ Prohibit access to Control Panel and PC settings

Set to: Enabled

Result:

Evan → blocked from Control Panel / PC Settings

LabAdmin → not affected

The distinction is important because **LabAdmin** is not a member of the **Lab-Users** security group.

48. Group Policy Security Filtering Troubleshooting

The most significant Windows troubleshooting exercise involved **Lab-User-Policy**.

Security filtering was changed from: Authenticated Users

To: Lab-Users

The Lab-Users group had:

Read
Apply Group Policy

However, the policy initially failed to apply.

gpresult reported:

Lab-User-Policy
Filtering: Not Applied (Unknown Reason)

At first, the obvious configuration appeared correct:

- Correct user OU
- Correct GPO link
- Link enabled
- GPO enabled
- Correct security group membership
- Read permission
- Apply Group Policy permission

The missing piece was that the **computer account also needed Read access to the GPO.**

The following permission was added:

Domain Computers
└─ Read

Domain Computers was deliberately **not** given Apply Group Policy.

Final relevant permissions:

Lab-Users
└─ Read
└─ Apply Group Policy

Domain Computers
└─ Read

After forcing a Group Policy refresh:

gpupdate /force

the policy successfully applied.

Verification:

Evan

→ Control Panel blocked

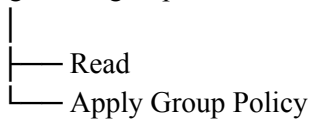
LabAdmin

→ Control Panel accessible

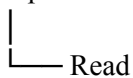
49. Group Policy Troubleshooting Lesson

A useful troubleshooting model was learned for user-targeted GPOs:

Target user/group



Computer account



The computer needs to be able to retrieve the GPO, while the targeted user/group determines whether the policy actually applies.

This was a practical example of why Group Policy troubleshooting cannot stop at simply checking whether a GPO is linked.

A systematic troubleshooting process includes checking:

1. User OU placement
2. Computer OU placement
3. GPO link
4. Link enabled status
5. GPO enabled status
6. Security filtering
7. Read permissions
8. Apply Group Policy permissions
9. User/group membership
10. Domain authentication
11. Policy refresh/logon requirements
12. **gpresult** output

50. Group Policy Refresh

The command: `gpupdate /force`

was used during testing.

An important concept learned is that **gpupdate /force** is **not normally required every time an administrator changes a GPO**.

In a domain environment:

Administrator changes GPO

↓

GPO stored in Active Directory / SYSVOL

↓

Domain computers periodically refresh

↓

Clients retrieve updated policy

↓

Policy applies

gpupdate /force is particularly useful for:

- Immediate testing
- Troubleshooting
- Avoiding the normal refresh wait

Some policies additionally require logoff/logon or a restart.

51. Active Directory Account Troubleshooting

A complete account recovery exercise was performed using the **Evan** domain account.

Step 1 — Disable Account

The account was disabled through Active Directory Users and Computers.

A fresh authentication attempt on CLIENT01 was rejected because the account was disabled.

An initial test appeared to allow login because an already-established session/cached authentication was involved.

A fresh login correctly demonstrated the disabled-account behavior.

Step 2 — Reset Password

The user's domain password was reset through ADUC.

An important distinction was confirmed:

Resetting a password does not automatically re-enable a disabled account.

Step 3 — Re-enable Account

The account was re-enabled through the user's Account properties.

Step 4 — Verify Authentication

CLIENT01 successfully accepted the new password and Evan was able to authenticate again.

This completed the workflow:

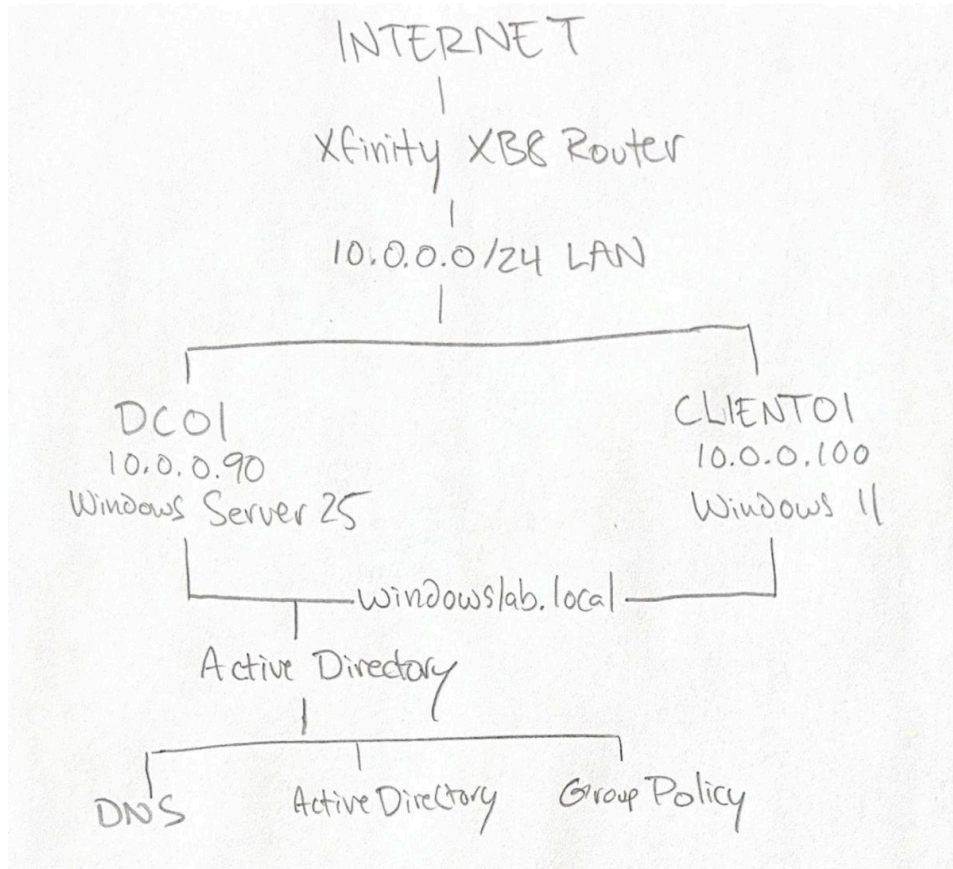
Disable account
↓
Verify authentication failure
↓
Reset password
↓
Re-enable account
↓
Authenticate successfully

Concepts demonstrated:

- Disabling domain accounts
 - Enabling domain accounts
 - Administrator password resets
 - Domain authentication
 - Cached credentials
 - Basic account troubleshooting
 - Difference between password state and account enabled/disabled state
-

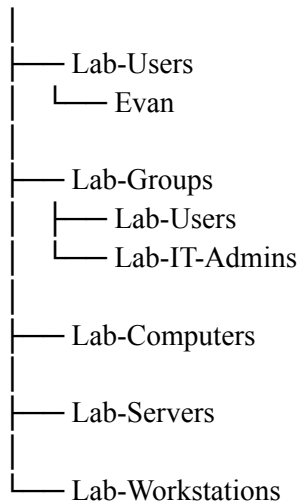
52. Current Windows Lab Architecture

Overall Architecture



AD Structure

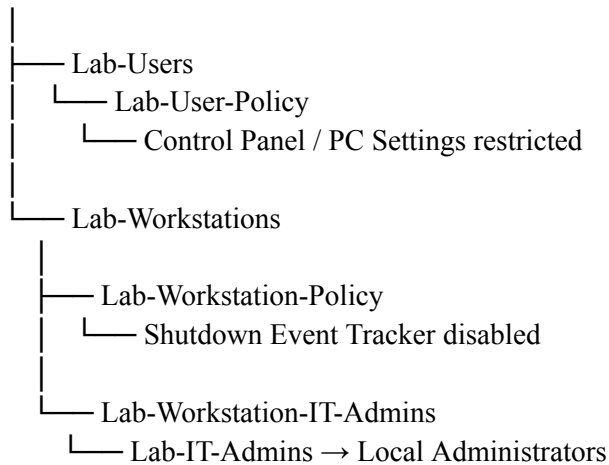
windowslab.local



└─ CLIENT01

Group Policy Structure

windowslab.local



53. Major Technical Concepts Learned

Virtualization

- Bare-metal hypervisors
- Proxmox VE
- VMs vs. LXC's
- Virtual CPUs
- Virtual RAM
- Virtual disks
- UEFI/OVMF
- q35 virtual hardware
- VirtIO devices
- Linux bridges
- LVM
- Proxmox storage pools

Networking

- IPv4 addressing
- /24 subnetting
- Default gateways

- Static addressing
- DHCP reservations
- DNS
- Local DNS
- DNS filtering
- DNS troubleshooting
- Network routing
- Subnet routing
- VPNs
- Tailscale
- TCP/UDP ports
- Reverse proxies
- HTTP vs. HTTPS

Linux

- Ubuntu Server
- Debian
- SSH
- SSH keys
- Linux permissions
- Package management
- LVM
- CA certificate stores
- Service management

Docker

- Docker Engine
- Docker Compose
- Containers
- Images
- Volumes
- Bind mounts
- Docker networks
- Container permissions
- Docker socket
- Persistent application data
- Containerized services

Web Infrastructure

- HTML

- CSS
- Responsive design
- Nginx
- Caddy
- Reverse proxying
- HTTPS
- Internal certificate authorities
- TLS troubleshooting

Monitoring

- Uptime Kuma
- Service availability monitoring
- DNS troubleshooting
- Certificate troubleshooting
- Container-level certificate trust
- Node.js certificate trust

Windows / Enterprise Administration

- Windows Server
 - Windows 11
 - Active Directory
 - AD DS
 - Domain controllers
 - DNS
 - Organizational Units
 - Security groups
 - Domain users
 - Domain joining
 - Group Policy
 - GPO security filtering
 - Group Policy Preferences
 - Local administrator management
 - **gpupdate**
 - **gpresult**
 - Account disabling/enabling
 - Password resets
 - Authentication troubleshooting
-

54. Most Valuable Troubleshooting Experiences

Several parts of the project provided particularly useful hands-on troubleshooting experience.

VirtIO Network Driver

Windows initially failed to recognize the virtual network adapter.

The appropriate VirtIO driver was located and installed, restoring network functionality.

Windows DNS

CLIENT01 initially used Xfinity IPv6 DNS servers instead of DC01.

The issue was identified by examining DNS configuration and **nslookup** behavior.

IPv6 was disabled in the lab, allowing DNS traffic to consistently use DC01.

Nextcloud Trusted Domain

Nextcloud rejected its **.homelab** hostname.

The hostname was added to the trusted domain configuration.

Caddy Internal CA

Applications initially encountered certificate validation errors because Caddy's private CA was not trusted.

The CA was installed on the necessary systems and devices.

Uptime Kuma Container TLS

The host trusted Caddy's certificate, but the Docker container and Node.js runtime did not automatically inherit the same trust.

The certificate was mounted into the container and explicitly provided to Node.js.

Group Policy Filtering

Lab-User-Policy appeared correctly configured but was reported by **gpresult** as:

Filtering: Not Applied (Unknown Reason)

The missing **Domain Computers** → **Read** permission was identified and corrected.

This was the most directly relevant enterprise troubleshooting exercise in the Windows portion of the lab.

55. Current Service Inventory

Component	Type	Address / Port	Purpose
Proxmox	Bare-metal hypervisor	10.0.0.50:8006	Virtualization
Ubuntu	VM 100	10.0.0.60	Docker host
Tailscale	LXC 101	10.0.0.70	VPN/subnet routing
Pi-hole	LXC 102	10.0.0.80	DNS/ad blocking
DC01	Windows VM	10.0.0.90	AD/DNS
CLIENT01	Windows VM	10.0.0.100	Domain workstation
Jellyfin	Docker	8096	Movie streaming
Navidrome	Docker	4533	Music streaming
Nextcloud	Docker	8080	File storage/journaling
Kavita	Docker	5000	Book/PDF library
Portainer	Docker	9443	Docker management
Caddy	Docker	80/443	Reverse proxy/HTTPS
Uptime Kuma	Docker	3001	Service monitoring
Dashboard/Nginx	Docker	8081	Homelab dashboard

56. Application URLs

The normal application interface is intentionally based on friendly hostnames rather than IP addresses and ports:

<https://dashboard.homelab>

https://jellyfin.homelab
https://kavita.homelab
https://nextcloud.homelab
https://navidrome.homelab
https://portainer.homelab
https://uptime.homelab
http://pi-hole.homelab

The **.homelab** names are resolved by Pi-hole and routed through Caddy.

57. Overall Architecture

The complete homelab can be viewed as several interconnected layers:

PHYSICAL LAYER:

Dell OptiPlex 5080 Micro
i5-10500T / 32 GB RAM / 512 NVMe

VIRTUALIZATION LAYER:

Proxmox VE
vmbr0 / local-lvm
Ubuntu VM / Tailscale LXC / Pi-hole LXC / Windows VMs

APPLICATION LAYER:

Docker / Compose
Jellyfin / Navidrome / Nextcloud / Kavita
Portainer / Caddy / Kuma / Nginx

NETWORK SERVICES:

Pi-hole DNS
Tailscale VPN
Caddy HTTPS / reverse proxy
Uptime Kuma monitoring

WINDOWS ENTERPRISE LAB:

DC01 / AD DS / DNS / GPO
CLIENT01 / Domain Authentication

58. Design Principles

Several design principles were intentionally followed throughout the project.

Keep Infrastructure Separate

Network infrastructure services such as Pi-hole and Tailscale run separately from application workloads.

Keep Applications Containerized

Linux applications are deployed as Docker containers so they remain isolated from the underlying Ubuntu operating system.

Use Persistent Storage

Application data and media are stored outside disposable container filesystems through Docker volumes and bind mounts.

Keep Remote Access Private

The homelab is not publicly exposed.

Remote access is provided through Tailscale rather than router port forwarding.

Separate DNS, Routing, and HTTPS

These are treated as different infrastructure functions:

Pi-hole → DNS

Tailscale → VPN/routing

Caddy → HTTPS/reverse proxy

Understanding those boundaries made troubleshooting substantially easier.

Use Groups Instead of Individual Permissions

The Windows lab uses security groups such as **Lab-IT-Admins** rather than assigning administrator privileges individually wherever possible.

59. Future Improvements

The current lab is functional, but several improvements remain planned.

Dedicated Storage

The current Ubuntu VM's virtual disk is sufficient for the proof-of-concept media library but is not appropriate for a large long-term media collection.

Planned architecture:

NVMe / VM storage

└── OS / Docker / applications

Large physical HDD

└── Movies / Music / Books / bulk data

Backups

A dedicated backup strategy has not yet been fully implemented.

Future work should include backups for:

- Docker Compose configurations
- Application data
- Nextcloud
- Proxmox VMs
- Windows lab configuration where appropriate

60. Final Project Status

The homelab has progressed from a bare Dell OptiPlex into a functioning multi-layer IT environment.

Infrastructure

- Dell OptiPlex virtualization host
- 32 GB RAM upgrade
- Proxmox VE
- Static host addressing
- DHCP reservation

- Proxmox storage
- vmbr0 virtual networking

Linux

- Ubuntu Server
- Debian LXC's
- SSH
- SSH key authentication
- LVM
- Linux administration

Docker

- Docker Engine
- Docker Compose
- Portainer
- Persistent volumes
- Bind mounts
- Container networking

Applications

- Jellyfin
- Navidrome
- Nextcloud
- MariaDB
- Kavita
- Caddy
- Uptime Kuma
- Custom Nginx dashboard

Networking

- IPv4 configuration
- Static addressing
- DNS
- Pi-hole
- DNS filtering
- Local DNS records
- Tailscale
- Subnet routing
- Remote access testing
- Internal HTTPS

- Reverse proxy

Windows / Active Directory

- Windows Server 2025
 - Windows 11
 - Active Directory Domain Services
 - Domain controller
 - DNS
 - **windowslab.local**
 - Organizational Units
 - Security groups
 - Domain users
 - Domain joining
 - Domain authentication
 - Computer-targeted GPOs
 - User-targeted GPOs
 - GPO security filtering
 - Group Policy Preferences
 - Local administrator management
 - **gpupdate**
 - gpresult
 - Account disabling/enabling
 - Password reset workflow
 - GPO troubleshooting
-

61. Project Summary

The homelab now represents a small, functional IT environment rather than a collection of isolated experiments.

A single physical server provides the virtualization layer, with Proxmox hosting Linux infrastructure, Docker applications, VPN/DNS services, and a Windows enterprise lab.

The Linux side demonstrates:

Virtualization → Linux → SSH → Docker → Compose → Applications → DNS → VPN → HTTPS → Monitoring

The Windows side demonstrates:

Windows Server → Active Directory → DNS → Users/Groups/OUs → Domain Join → Group Policy → Security Filtering → Workstation Administration → Account Troubleshooting

The most valuable aspect of the project is not simply the number of services deployed. It is the ability to **configure, integrate, test, break, troubleshoot, and explain the individual layers of the system.**

The lab has provided hands-on experience with the same categories of infrastructure encountered in entry-level IT and systems administration environments: endpoint configuration, virtualization, networking, DNS, Linux administration, Windows administration, authentication, permissions, centralized policy, service deployment, monitoring, and troubleshooting.